

Лабораторная работа №2. TypeScript. Создание системы для каталога товаров

Описание задачи

Разработайте систему для управления каталогом товаров с использованием TypeScript. Вам нужно создать классы для каталога товаров и товаров, а также реализовать несколько дополнительных классов для улучшения функциональности.

Требования

1. Класс `Product`

Создайте класс `Product`, который будет представлять товар. Класс должен содержать следующие свойства:

- `id`— уникальный идентификатор товара.
- `name`— название товара.
- `price`— цена товара.
- `description`— описание товара (опциональное).
- `category`— категория товара (используйте `enum ProductCategory`).

Методы:

- Конструктор для инициализации всех свойств.
- Метод `getInfo()`, который возвращает информацию о товаре в виде строки.

Используйте утилитные типы TypeScript для создания более гибких типов для ваших функций и классов.

- Используйте `Partial<Product>` для создания новых товаров с необязательными свойствами.
- Используйте `Omit<Product, 'id'>` для создания типа товара без идентификатора.
- Используйте `Pick<Order<Product>, 'id' | 'totalPrice'>` для получения сводной информации о заказе.

2. Класс `Catalog`

Создайте класс `Catalog`, который будет представлять каталог товаров. Класс должен содержать следующее:

- Свойство `products`— массив товаров типа `Product`.

Методы:

- Конструктор для инициализации пустого массива товаров.
- Метод `addProduct(product):` добавляет новый товар в каталог.
- Метод `removeProduct(id):` удаляет товар по его идентификатору.
- Метод `getProductById(id):` возвращает товар по его идентификатору.

- Метод `getAllProducts()` : возвращает все товары в каталоге.
- Метод `getProductsByCategory(category)` : возвращает все товары в указанной категории.

3. Класс `Order`

Создайте дженерик-класс `Order<T extends Product>` для представления заказа. Класс должен содержать:

- `id`— уникальный идентификатор заказа.
- `products`— массив товаров в заказе (дженерик `Array<T>`).
- `totalPrice`— общая стоимость заказа.

Методы:

- Конструктор для инициализации всех свойств.
- Метод `calculateTotalPrice()`, который вычисляет общую стоимость заказа.
- Метод `getOrderInfo()`, который возвращает информацию о заказе в виде строки.

4. Класс `Customer`

Создайте класс `Customer`, который будет представлять покупателя. Класс должен содержать:

- `id`— уникальный идентификатор покупателя.
- `name`— имя покупателя.
- `email`— электронная почта покупателя.

Методы:

- Конструктор для инициализации всех свойств.
- Метод `getCustomerInfo()`, который возвращает информацию о покупателе в виде строки.

5. Класс `OrderManager`

Описание

Класс `OrderManager` будет отвечать за управление заказами, их созданием, поиском и организацией. Это позволит отделить логику управления заказами от остальных классов, таких как `Product`, `Catalog`, и `Order`.

Свойства

- `orders`— массив для хранения заказов (тип `Array<Order<Product>>`).

Методы

- Конструктор для инициализации пустого массива заказов.
- `createOrder(customer, products)`:

- Метод для создания нового заказа. Принимает клиента и массив товаров, создаёт и возвращает новый объект заказа.
- `getOrderByid(id):`
 - Метод для поиска заказа по его идентификатору.
- `getAllOrders():`
 - Метод для получения всех заказов.
- `getOrdersByCustomer(customerId):`
 - Метод, который возвращает все заказы, сделанные конкретным покупателем.

Вопросы:

1. Создание классов в TS?
2. Директивы доступа?
3. Что такое утилитные типы в TypeScript?
4. Перечислить существующие утилитные типы и их назначение.
5. Что такое `enum` в TypeScript?
6. Что такое дженерики в TypeScript?